How-To: Lifecycle Hooks

ragent hooks let you run shell commands at key points in the session lifecycle. This guide covers the two mechanisms for controlling tool execution: **PreToolUse** and **PostToolUse** hooks.

Table of Contents

Configuration

Hooks are defined in the `hooks` array of `ragent.json`:

```json
{
  "hooks": [
    {
      "trigger": "pre_tool_use",
      "command": "/path/to/my-pre-tool-hook.sh",
      "timeout_secs": 30
    },
    {
      "trigger": "post_tool_use",
      "command": "/path/to/my-post-tool-hook.sh",
      "timeout_secs": 10
    }
  ]
}
```

Each hook entry has three fields:

Field	Type	Default	Description
`trigger`	string	—	When to fire (see triggers)
`command`	string	—	Shell command to execute (runs via `sh -c`)
`timeout_secs`	integer	30	Timeout in seconds. Enforced for `post_tool_use` and the fire-and-forget triggers; **not** enforced for `pre_tool_use` (see PreToolUse Hooks).

Invalid hook entries (for example an unknown `trigger` string) are skipped at load time with a log warning — no startup error surfaces to the user.

PreToolUse Hooks

PreToolUse hooks fire **before** a tool is executed. They can approve, deny, modify, block, or warn about the upcoming tool call.

Two mechanisms control tool execution, and they work side by side:

1. **JSON-Decision Protocol** — the hook writes a JSON decision to stdout.
2. **Exit-Code Convention** — the hook's exit code signals block / warn / error.

JSON-Decision Protocol

When the hook exits with code **0**, ragent parses stdout as JSON. The following decisions are recognised:

stdout JSON	Effect
`{"decision": "allow"}`	Skip the UI permission prompt and allow the tool.
`{"decision": "deny", "reason": "..."}`	Deny the tool call with an optional reason.
`{"modified_input": {"path": "new/path"}}`	Replace the tool's input arguments.
(empty or invalid JSON)	No decision — normal permission flow applies.

Example — allow a specific tool:

```
#!/bin/sh
# Allow all read operations without prompting
echo '{"decision": "allow"}'
```

Example — deny with a reason:

```
#!/bin/sh
# Block writes to /etc
echo '{"decision": "deny", "reason": "Writes to /etc are not allowed"}'
```

Example — modify input arguments:

```
#!/bin/sh
# Redirect all file reads to a sandbox directory
echo '{"modified_input": {"path": "/sandbox/src/main.rs"}}'
```

Exit-Code Convention

When the hook exits with a **non-zero** status code, ragent interprets the exit code **before** parsing stdout JSON:

Exit code	Behaviour
0	Parse stdout JSON (see JSON-Decision Protocol).

Exit code	Behaviour
1	**Warn** — allow the tool to proceed, but emit `tracing::warn!` and publish `Event::HookWarning` on the event bus so the TUI can surface it.
2	**Block** — the tool call is blocked. The hook's stderr (trimmed, capped at 500 characters) is used as the block reason. Stdout JSON is **ignored**.
≥ 3	**Error** — treat as a hook failure (not a policy decision). Fall through to the normal permission flow with a `tracing::error!` diagnostic.

Example — block with exit code 2:

```
#!/bin/sh
# Block all bash commands that contain "rm -rf"
if echo "$RAGENT_TOOL_INPUT" | grep -q "rm -rf"; then
  echo "Blocked: destructive command detected" >&2
  exit 2
fi
# Otherwise, allow without prompting
echo '{"decision": "allow"}'
```

Example — warn with exit code 1:

```
#!/bin/sh
# Warn when a write tool is used outside src/
if echo "$RAGENT_TOOL_INPUT" | grep -qv '"path":.*"src/'; then
  echo "Warning: write outside src/ directory" >&2
  exit 1
fi
exit 0
```

Example — hook error with exit code 3:

```
#!/bin/sh
# This hook has a bug (missing dependency)
if ! command -v jq >/dev/null 2>&1; then
  echo "Hook error: jq is not installed" >&2
  exit 3
fi
# Normal processing would go here
echo '{"decision": "allow"}'
```

When a hook exits with code ≥ 3, the tool call is **not** blocked — it falls through to the normal permission flow so a broken hook cannot lock the user out of all tool use.

Precedence Rules

1. **Exit code 2 takes absolute precedence** over stdout JSON. Even if the hook writes `{"decision": "allow"}` to stdout, exit code 2 will block the tool call.
2. **Exit code 1 does not parse stdout JSON.** The warning is emitted regardless of stdout content.
3. **Multiple hooks are evaluated in order.** The first hook that returns a decision (`allow`, `deny`, `modified_input`, or `Blocked`) wins; remaining hooks are not executed.

4. **Spawn failure** is treated as exit code ≥ 3 (hook error). The tool call falls through to the normal permission flow.

> **No timeout on the pre-hook path:** PreToolUse hooks are spawned synchronously via `std::process::Command::output()` with no timeout wrapper, so `timeout_secs` is silently ignored here — a hanging pre-hook blocks every tool call until it exits. `timeout_secs` *is* enforced for `post_tool_use` hooks and the fire-and-forget triggers.

PostToolUse Hooks

PostToolUse hooks fire **after** a tool has executed. They can modify the tool's output, warn about the result, or flag it as policy-violated.

Exit-Code Convention

Exit code	Behaviour
0	Parse stdout JSON for `modified_output` (see below).
1	**Warn** — emit `tracing::warn!` and publish `Event::HookWarning`. The tool result is not modified.
2	**Flag** — publish `Event::ToolResultFlagged` with stderr as the reason. The tool result is **not** suppressed (the call already executed), but the flag appears in the session log and TUI.
≥ 3	**Error** — treat as a hook failure. No effect on the tool result.

Modified Output

When a PostToolUse hook exits with code 0 and writes JSON to stdout, ragent looks for a `modified_output` field:

```json
{
  "modified_output": {
    "content": "Sanitised output here",
    "metadata": {"custom": "value"}
  }
}
```

If the `modified_output.content` field is present, it replaces the tool's output content.

Example — redact secrets from tool output:

```sh
#!/bin/sh
# Read the original output from the environment variable
OUTPUT="$RAGENT_TOOL_OUTPUT"
# Redact API keys (simplified example)
REDACTED=$(echo "$OUTPUT" | sed 's/sk-[a-zA-Z0-9]*/[REDACTED]/g')
# Return modified output
echo "{\"modified_output\": {\"content\": \"$REDACTED\"}}"
```

Example — flag suspicious output:

```sh
#!/bin/sh
# Check if the tool output contains sensitive data
if echo "$RAGENT_TOOL_OUTPUT" | grep -q "password"; then
  echo "Flagged: output contains potential password" >&2
  exit 2
fi
exit 0
```

Other Hook Triggers

In addition to `pre_tool_use` and `post_tool_use`, ragent supports these triggers:

Trigger	Description
`on_session_start`	Fired when a session receives its first user message.
`on_session_end`	Fired after a session completes processing a user message.
`on_error`	Fired when an LLM call or tool execution returns an error.
`on_permission_denied`	Fired when a tool call is rejected due to a permission rule.
`on_turn_start`	Fired at the start of each agent turn/iteration.
`on_turn_end`	Fired at the end of each agent turn/iteration.
`on_compaction`	Fired when context compaction is performed.

These triggers fire asynchronously — errors are logged but never fatal. The `RAGENT_ERROR` environment variable is set for `on_error` hooks.

Environment Variables

All hooks receive these environment variables:

Variable	Description
`RAGENT_TRIGGER`	The trigger name (e.g. `pre_tool_use`)
`RAGENT_WORKING_DIR`	The session working directory

`on_error` hooks additionally receive:

Variable	Description
`RAGENT_ERROR`	The error message

PreToolUse hooks additionally receive:

Variable	Description
`RAGENT_TOOL_NAME`	The name of the tool being invoked
`RAGENT_TOOL_INPUT`	JSON string of the tool arguments

PostToolUse hooks additionally receive:

Variable	Description
`RAGENT_TOOL_NAME`	The name of the tool that was invoked
`RAGENT_TOOL_INPUT`	JSON string of the tool arguments
`RAGENT_TOOL_OUTPUT`	JSON string of the tool output
`RAGENT_TOOL_SUCCESS`	`"true"` or `"false"`

Turn and compaction triggers additionally receive:

Variable	Description
`RAGENT_TURN_NUMBER`	Current agent turn/iteration number (`on_turn_start`, `on_turn_end`)
`RAGENT_COMPACTION_REASON`	Why compaction fired (`on_compaction`)
`RAGENT_COMPACTION_TOKENS_BEFORE`	Token count before compaction (`on_compaction`)
`RAGENT_COMPACTION_TOKENS_AFTER`	Token count after compaction (`on_compaction`)

Complete Example

Here is a `ragent.json` snippet that configures both PreToolUse and PostToolUse hooks:

```json
{
  "hooks": [
    {
      "trigger": "pre_tool_use",
      "command": ".ragent/hooks/pre-tool-use.sh",
      "timeout_secs": 10
    },
    {
      "trigger": "post_tool_use",
      "command": ".ragent/hooks/post-tool-use.sh",
      "timeout_secs": 10
    }
  ]
}
```

`.ragent/hooks/pre-tool-use.sh`:

```sh
#!/bin/sh
# PreToolUse hook: enforce security policies
```

```sh
TOOL="$RAGENT_TOOL_NAME"
INPUT="$RAGENT_TOOL_INPUT"

# Block bash commands that contain "rm -rf /"
if [ "$TOOL" = "bash" ] && echo "$INPUT" | grep -q "rm -rf /"; then
  echo "Blocked: refused to execute destructive command" >&2
  exit 2
fi

# Warn about writes outside the project directory
if [ "$TOOL" = "write" ] && ! echo "$INPUT" | grep -q '"path":.*"src/'; then
  echo "Warning: write target is outside src/" >&2
  exit 1
fi

# Allow all other tools without prompting
echo '{"decision": "allow"}'
```

`.ragent/hooks/post-tool-use.sh`:

```sh
#!/bin/sh
# PostToolUse hook: audit tool results

OUTPUT="$RAGENT_TOOL_OUTPUT"

# Flag outputs that contain potential secrets
if echo "$OUTPUT" | grep -qE "(sk-[a-zA-Z0-9]{20,}|AKIA[A-Z0-9]{16})"; then
  echo "Flagged: output contains potential API key" >&2
  exit 2
fi

# No modification needed
exit 0
```

Summary

Mechanism	PreToolUse	PostToolUse
Exit 0	Parse stdout JSON for decision	Parse stdout JSON for `modified_output`
Exit 1	Warn + allow (publish `HookWarning`)	Warn (publish `HookWarning`)
Exit 2	Block (publish denial to agent)	Flag (publish `ToolResultFlagged`)
Exit ≥ 3	Hook error → normal permission flow	Hook error → no effect on result
Spawn failure	Treated as exit ≥ 3	Treated as exit ≥ 3
Timeout	Not enforced (no timeout wrapper)	Treated as exit ≥ 3 (`timeout_secs`)

When multiple PostToolUse hooks fire, the aggregated outcome follows the precedence **Flagged > Warn**

> **Ok**, and among Ok results the last `modified_output` wins.

The two mechanisms (JSON decisions and exit codes) are **complementary**: JSON decisions provide fine-grained control (allow / deny / modify), while exit codes provide a simple, language-agnostic way to signal block, warn, or error from any scripting language.